

Revision of Date, Time and TimeStamp in JDBC

13 September 2026 08:52

java.sql.Date: The `java.sql.Date` extends `java.util.Date` class. The `java.sql.Date` is used in the JDBC API, as a wrapper around `java.util.Date` that handles SQL specific requirements. The class exists to represent SQL date which keeps year, month and days with no time. **java.sql.Date** should be used only when working with databases, like to set a date on a `java.sql.PreparedStatement`. Get a date from a `java.sql.ResultSet`, Working with `java.sql.SQLDATA` etc.

We can instantiate a `java.sql.Date` object using the following constructor:

- `Date(long date)`: Constructs a date object using the given milliseconds time value. For example:
WAP to create a `java.sql.Date` object and print the time in milliseconds and the `java.sql.Date` object.

```
public class DateandTime {
    public static void main(String[] args) {
        //java.util.Date date = new java.util.Date();
        long now = System.currentTimeMillis();
        Date date = new Date(now);
        System.out.println("Time in milliseconds: "+now);
        System.out.println("sql date: "+date);

        System.out.println("time in milliseconds using sql.Date object:
"+date.getTime());

    }
}
```

java.util.Date
↑
java.sql.Date

Output:

```
Time in milliseconds: 1789272168095
sql date: 2026-09-13
time in milliseconds using sql.Date object: 1789272168095
```

Or, you can use static `valueOf(...)` method to get an instance of `java.sql.Date` object.

static Date valueOf(String s): Convert the string in JDBC date escape format to a date value.

static Date valueOf(LocalDate date): Obtains an instance of `Date` from a local `Date` object with the same year, month, and day of the month value as given **LocalDate**.

For example: Use the above methods to print the Date.

```
package MySQLPrograms;

import java.sql.Date;
import java.time.LocalDate;
```

```
import java.time.Month;

public class SQLDateValueOfExample {
    public static void main(String[] args) {
        Date sqldate1 = Date.valueOf("1983-11-14");
        System.out.println("sql Date 1: "+sqldate1);

        Date sqldate2 = Date.valueOf(LocalDate.of(2004, Month.NOVEMBER, 30));
        System.out.println("sql Date 2: "+sqldate2);
    }
}
```

Output:

```
sql Date 1: 1983-11-14
sql Date 2: 2004-11-30
```

java.sql.Date Methods

- void setTime(long time): Set an existing Date object using the given millisecond time value.
- LocalDate toLocalDate(): Creates a local date instance using the year, month and date from this Date object.
- String toString(): Format the date in the Date escape format "YYYY-MM-DD".

java.time.Instant

An instantaneous point on the time-line.

This class models a single instantaneous point on the time-line. This might be used to record event time-stamps in the application.

The range of an instant requires the storage of a number larger than a **long**. To achieve this, the class stores a **long** representing epoch-seconds and an **int** representing nanosecond-of-second, which will always be between 0 and 999,999,999. The epoch-seconds are measured from the standard Java epoch of **1970-01-01T00:00:00Z** where instants after the epoch have positive values, and earlier instants have negative values. For both the epoch-second and nanosecond parts, a larger value is always later on the time-line than a smaller value.

InstantiationError	java.lang
InstantiationException	java.lang
Instant	java.time
InstantSource	java.time
Instantiation	java.security.DrbgParameters
Instrumentation	java.lang.instrument
InstalledProviders	oracle.jdbc.driver.resource
InstanceAlreadyExistsException	javax.management
InstanceNotFoundException	javax.management
InstanceProperty	oracle.jdbc.internal.OracleConn...
InstantValueEncoder	com.mysql.cj.protocol.a
Instrument	javax.sound.midi

← not present in java.sql.Date

```
package MyJDBCOne;
import java.sql.Date;
import java.time.Instant;
import java.time.LocalDate;
public class SQLDateMethods {
    public static void main(String[] args) {
        long now = System.currentTimeMillis();
        Date sqlDate = new Date(now);
        System.out.println("SQLDate: " + sqlDate);
        sqlDate.setTime(356373000000L);
        System.out.println("sqlDate: " + sqlDate);
        LocalDate localDt = sqlDate.toLocalDate();
    }
}
```

```
System.out.println("LocalDate: "+ localDt);  
Instant instant = sqlDate.toInstant();  
System.out.println("Instant: "+ instant);
```

```
}  
}
```

Output:

SQLDate:2026-09-13

sqlDate: 1981-04-17

LocalDate: 1981-04-17

Exception in thread "main" java.lang.UnsupportedOperationException
at java.sql/java.sql.Date.toInstant(Date.java:316)
at MyJDBCOne.SQLDateMethods.main(SQLDateMethods.java:17)

*Instant cannot be used with
java.sql.Date .*

